RC-Gate — User Guide

Reliability-Conditioned Gated Mixture-of-Experts for claim-type and model-aware hallucination detection

Prathmesh P. Desai

October 6, 2026

1. What RC-Gate is
2. The three hypotheses
3. How the system works
    3.1 Pipeline
    3.2 Model specification
4. The signal table contract
5. Variants: baselines, controls, and the proposed method
6. Installation and quickstart
7. The milestones
8. Results
    8.1 Synthetic data (5 tables x 5 seeds, mean test AUROC)
    8.2 The strongest result: unseen (inverted) generator
    8.3 Real data: RAGTruth (800 claims x 3 seeds)
    8.4 Which benchmarks can and cannot run
9. The glass-box demo
10. Repository layout and documentation
11. Constraints the build honours
12. Limitations and honest negatives
13. CLI reference
14. Regenerating this guide

1. What RC-Gate is

Large language models hallucinate: they state things that are not true, fluently and with confidence. Detecting this from the *outside* — without looking inside the model, using only the text it produces — is the problem RC-Gate addresses.

No single external signal is reliable for every kind of claim:

- **Cross-model consensus** (asking several different LLMs the same question and comparing their answers) works well for claims about

entities and facts, but fails on numbers, because models trained on similar data tend to copy the same wrong figure from the same source.
- **Retrieval-based evidence** (checking a claim against documents) works well for numeric and extractive claims, but is weak for open-ended paraphrases.

RC-Gate's idea: **the reliability of each signal family is conditional on the kind of claim and on which generator produced it —** so *learn that condition* and route each claim to the family it deserves. A tiny neural **gate** (229 parameters, conditioned only on claim type and generator identity) assigns trust weights `w_cross` and `w_evid` to two expert scorers; a stacked meta-learner fuses their outputs into a hallucination probability.

Crucially, the system is **glass-box**: every prediction ships with the trust weights and a one-line explanation of which signal family was trusted and why. The weights are logged during training and evaluation, never hidden behind a bare probability.

Status: research prototype built for a B.Tech project review. Synthetic results validate the *machinery*; the first real-data result is negative and is reported as measured (Section 8).

2. The three hypotheses

The project tests three hypotheses from the proposal:

#	Hypothesis	How the project tests it
H1	Signal reliability is conditional on claim type and generator	Per-(type, generator) reliability tables; ablations A5 (gate on type only) and A6 (gate on generator only)
H2	The gated model beats flat fusion and naive voting, especially on numeric/extractive claims	Paired bootstrap of A7 vs A0 per claim type; A3 (static blend) and A4 (flat GBM) as controls
H3	Gate weights are interpretable	Weights returned by every inference call, logged per evaluation, rendered by the demo

Honest summary as of this guide: H1 holds strongly for the claim-type half on synthetic data and barely for the generator half; H2 holds on synthetic data (numeric slice) but **fails on the first real benchmark** (Section 8); H3 holds mechanically everywhere — and on real data the weights are near-uniform, which is itself the diagnosis (the metadata carries no signal there).

3. How the system works

3.1 Pipeline

1. **Generate** — a question goes to several LLMs (all calls disk-cached and budget-capped).
2. **Extract** — each answer is split into atomic claims (sentences), and pronouns are replaced with their referents (`dealias`) using the question and prior sentences.
3. **Cross-model signals** `s_cross = (agree, contra, sem_dist)` — for each claim, the *other* generators' answers are asked whether it holds (via NLI): `agree` = fraction supporting, `contra` = max contradiction probability, `sem_dist` = 1 minus mean cosine similarity to the other answers' best-matching sentences. Rows with no other generator get neutral 0.5/0.5/1.0 and are flagged via `cross_n_others`.
4. **Evidence signals** `s_evid = (support, src_quality, conflict)` — BM25 retrieval over the benchmark's shipped context, NLI for support/conflict, `src_quality` as a documented heuristic. For numeric claims a deterministic **arithmetic checker** verifies parseable expressions exactly and folds the verdict into `support`; it refuses to guess anything it cannot parse.
5. **Meta-features** `x_meta = (claim_type, generator_id)` — syntax-derived claim type and which generator produced the answer.
6. **Gate** — a small MLP on `x_meta` only, softmax outputs `w_cross + w_evid = 1`.
7. **Two experts** — one linear scorer per signal family.
8. **Meta-learner** — fuses the gate-weighted expert logits into the final probability.
9. **Report** — per claim: probability, both trust weights, both expert logits, and a one-line explanation.

3.2 Model specification

```
x_meta = [Emb_type(claim_type) | Emb_gen(generator_id)]
# 4-d + 4-d = 8-d
gate   = MLP(8 -> 16 -> 2), ReLU
w      = softmax(gate(x_meta) /
tau)                          # tau = 1.0, fixed
z_c    = Linear(3 -> 1)
(s_cross)                               # cross expert logit
z_e    = Linear(3 -> 1)
(s_evid)                                # evidence expert
logit
logit  = Linear(2 -> 1)([w_c * z_c, w_e *
z_e])              # logits fused, not probs
P      = sigmoid(logit)
```

Design rules that make the method what it is (full reasoning in `docs/adr/`):

- **The gate sees metadata only** (ADR-001). No signal value ever reaches the gate — enforced by two unit tests that fail the build if

anyone wires signals into it. This is what keeps the trust weights meaningful and makes A4 a clean control.
- **Experts are fused at the logit level** (ADR-002). Probability fusion shrinks every product toward zero; logit fusion preserves disagreement.
- **Four claim types** (ADR-003): 0 numeric, 1 entity, 2 extractive, 3 abstractive — matching the survey taxonomy and RAGTruth's label schema, assigned by a rule-based classifier (no LLM).
- Trained **end-to-end** with BCE, Adam, early stopping on validation AUROC; during training the generator id is replaced by a reserved `UNK` embedding with probability 0.1, so the cold-start path (unseen generator) is trained, not just hoped for.
- A7 has **229 trainable parameters** (A5 141, A6 149) against a 50k budget; everything runs on CPU.

4. The signal table contract

All modelling code reads one parquet table; extractors write it. This decouples expensive signal computation from fast experimentation.

column	type	meaning
`question_id`	str	split key (splits are grouped by question, never by claim)
`claim_id`	str	unique
`claim_text`	str	de-aliased claim
`benchmark`	str	`synthetic`, `ragtruth`, …
`claim_type`	int	0 numeric, 1 entity, 2 extractive, 3 abstractive
`generator_id`	int	index into `configs/generators.yaml`; reserved `UNK` for unseen
`agree`, `contra`, `sem_dist`	float	cross-model family, all in [0,1]
`support`, `src_quality`, `conflict`	float	evidence family, all in [0,1]
`label`	int	1 = hallucinated

5. Variants: baselines, controls, and the proposed method

All variants run from the same table and the same splits, so comparisons are apples-to-apples:

ID	Variant	Role
A0	Flat logistic regression on all 6 signals	naive fusion baseline the project must beat
A1	Cross-model signals only	BEACON-style
A2	Evidence signals only	Medico-style
A3	Static learned scalar blend of the two experts	tests whether *conditioning* matters, not just *learning*
A4	Flat gradient-boosted trees on 6 signals **plus** claim type and generator	**the critical control**: if A4 matches A7, the gate adds nothing beyond metadata access
A5	Gate on claim type only	isolates the claim-type contribution
A6	Gate on generator only	isolates the generator contribution
A7	Full RC-Gate	the proposed method
A8	Naive majority vote across judges	only runs if judge votes exist in the table

6. Installation and quickstart

```
pip install -e ".[dev,demo]"

make m0     # scaffold check: pytest green, ruff clean,
        CLIs answer
make m1     # build the synthetic signal table with
        planted structure
make m2     # train baselines A0-A4 (5 seeds)
make m3     # train RC-Gate A7 + ablations A3/A5/A6
make m4     # full evaluation on synthetic: AUROC/F1/ECE,
        bootstrap CIs,
            #   leave-one-generator-out, gate-weight
        heatmap
make m5
        # build the real RAGTruth signal table (cached
        extractors)
make m6     # full evaluation on RAGTruth
make m7     # glass-box demo, headless verification
```

```
make stages # run all of the above, M0 -> M7, in order
```

Each stage is the real command used at that milestone, and each is cache-aware: signal extraction is served from a disk cache (a second RAGTruth build takes 2.4 s vs 37.6 s), every run writes its config, git hash, wall-clock and API accounting next to its results, and API spend is $0.00 — NLI and embeddings run on local CPU.

7. The milestones

Milestone	What it delivers	Acceptance result
M0	Scaffold: config system, signal-table contract, cache, splits, CLIs	tests green, lint clean, `train --help` answers
M1	Synthetic table with *planted* conditional reliability (numeric: anti-informative cross family; inverted generator; 13.5k claims)	planted correlations have the intended signs per (type, generator)
M2	Baselines A0-A4 on synthetic	A0 numeric 0.478 (< 0.65 target) while entity 0.858 — gate met
M3	RC-Gate A7 + ablations A3/A5-A7	A7 numeric +0.377 over A0 (target +0.15); mean `w_evid` on numeric 0.809 (> 0.7); gate does not collapse
M4	Evaluation harness: per-slice AUROC/F1/precision/recall/ECE, bootstrap CIs, leave-one-generator-out, gate heatmap	one command produces the full report (~17 min CPU)
M5	Real signal extractors (claim extraction, NLI, BM25 evidence, arithmetic check) on RAGTruth	table builds end to end; second run fully cached in 2.4 s
M6	Real-data runs (M2-M4 pipeline on RAGTruth)	machinery runs end to end at $0.00; H1/H2 negative on this slice, reported as measured
M7	Glass-box Streamlit demo + headless mode	loads a trained checkpoint, never trains; verified live

8. Results

8.1 Synthetic data (5 tables x 5 seeds, mean test AUROC)

Slice	A0 flat LR	A3 static blend	A4 flat GBM + meta	A7 RC-Gate
numeric	0.478	0.470	**0.942**	0.855
entity	0.858	0.860	**0.927**	0.870
all	0.756	0.756	**0.929**	0.874

- A7 beats A0 on numeric by **+0.377** (95% CI [0.341, 0.414], paired bootstrap) — H2 supported on the numeric slice.
- Mean `w_evid` on numeric claims is **0.809**: the gate routes numeric claims to the evidence family, exactly as designed.
- **A4 beats A7 on every claim type.** This is reported, not tuned away: a GBM given claim type and generator can learn the per-cell reliability function directly. The gate's contribution is an interpretable inductive bias, not extra accuracy on planted data.

8.2 The strongest result: unseen (inverted) generator

Holding out generator 4 — the planted *inverted* family, whose agreement correlates positively with hallucination — and scoring only unseen-generator rows:

held-out generator	A0 AUROC	A7 AUROC
0-3 (normal)	0.74-0.78	0.89-0.92
4 (inverted, unseen)	**0.171**	**0.748**

A0 scores *below random* on the unseen inverted generator — the global coefficient it learned for `agree` has exactly the wrong sign there. A7 degrades gracefully and stays useful. This is the cleanest answer to "does it generalize to unseen generators?"

8.3 Real data: RAGTruth (800 claims x 3 seeds)

RAGTruth is the only M6-runnable benchmark: it ships six LLM answers per source document (the generator panel the cross-model family needs), human span labels, and the context text, with an MIT licence. Table: ~57 positives per seed (7.1%), 134 sources, all six generators; ~12,300 NLI calls on local CPU, $0.00 API spend.

Slice	A0 flat LR	A1 cross only	A4 flat GBM + meta	A7 RC-Gate
all	0.681	0.705	**0.740**	0.631

- Paired bootstrap, A7 - A0: **-0.050 [-0.210, 0.100]**, p = 0.415 — a loss with the null inside it.
- Gate weights sit near **0.45** across (type, generator) cells: the metadata carries little signal on this slice, so a correctly-functioning gate approaches uniform. Per the project's own rules, that is a finding, not a bug.
- Root causes, all checkable in the artefacts: (1) ~11 test positives make every CI roughly +/-0.2 wide; (2) RAGTruth hallucinations here are dominated by *subtle* conflicts — sentences that are retrievable and that other models do not contradict are near-invisible to black-box signals (the correlated-error failure the proposal itself flags); (3) val-tuned thresholds at a 7% positive rate give poor precision even when recall is high.
- Nothing was retuned after seeing these numbers. Full autopsy: `docs/postmortems/rcgate/002-m6-a4-beats-a7-on-real-data.md`.

8.4 Which benchmarks can and cannot run

benchmark	status	reason
RAGTruth	**run**	six generators per source, span labels, shipped context, MIT
HaluBench (incl. its DROP slice)	blocked	one answer per question (verified against the dataset parquet: no model field, no spans)
FinanceBench	blocked	one answer per question; single-document evidence
TruthfulQA / HaluEval-QA	blocked	open QA, no shipped context, one answer — evidence family would need a paid search source

Blocking is a policy, not an omission: inventing cross-model signals for single-answer datasets would put fabricated numbers into every table (ADR-005).

9. The glass-box demo

The demo (`rcgate/demo/app.py`) is the batch pipeline with a UI, not a parallel implementation — it uses the same claim extraction, typing and signal code as the real builds.

```
streamlit run rcgate/demo/app.py          # interactive UI
python -m rcgate.demo.app --config ragtruth \
    --question "..." --answer "..." [--context ...] [--
        other-answer ...]
```

- It **loads** the newest trained A7 checkpoint (`results/summary/<tag>/A7_seed*/model.*`) and refuses to train — numbers on stage must not depend on a split chosen live.
- Per claim it shows the probability, the stacked `w_cross` / `w_evid` trust split, and a one-line explanation.
- If you paste no other models' answers, the cross family is neutral (0.5/0.5/1.0) and the row is flagged — the demo never invents agreement to make the bars look interesting.
- Live behaviour matching the design: a numeric claim gets `w_evid` 1.00; an entity claim gets `w_cross` 0.92.

10. Repository layout and documentation

```
rcgate/                  package: data/, claims/, signals/,
models/, demo/ + CLIs
configs/                 one YAML per benchmark + planted
probabilities + budget
docs/adr/                6 architecture decision records
(Design Dungeons format)
docs/postmortems/rcgate/  3 postmortems: what broke, root
cause, what changed
tests/                   195 offline tests; the network is
never touched
results/summary/         committed summary tables + per-run
artefacts (configs, checkpoints)
Makefile                 m0–m7 stage targets + `make
stages`
DECISIONS.md             one line per judgment call, per
milestone
```

- **ADRs** record the load-bearing decisions: metadata-only gate (001), logit fusion (002), four claim types (003), RAGTruth span-overlap labelling (004), benchmark blocker policy (005), compute-cache split (006).
- **Postmortems** record the failures and their fixes: a stale signal table silently reused during parallel builds (001); the honest M6 negative (002); an explainer that named the wrong signal family — shipped because it had no tests, fixed with 13 regression tests (003).

- **DECISIONS.md** is the complete judgment-call log, one line each, with acceptance results per milestone.

11. Constraints the build honours

ID	Constraint	How it is honoured
C1	Black-box only	gate sees claim type + generator id; test-enforced
C2	CPU-only, <= 16 GB RAM	DeBERTa-v3-small NLI + MiniLM embeddings; A7 = 229 params
C3	Bounded API spend	single disk cache, `--dry-run`, `max_usd`/`max_calls`; local NLI cached separately (ADR-006)
C4	Reproducible	seeded runs, pinned stack, config + git hash written next to results
C5	Interpretable by construction	weights returned by every `predict()` and logged in every evaluation
C6	Leakage-free	splits grouped by `question_id`; scalers/thresholds/calibrators fit on train/val only
C7	Prototype scope	no fine-tuning, no scraping, no frameworks beyond the listed stack
C8	Honest reporting	synthetic results labelled as machinery validation; negatives reported as measured

Reproducibility in practice: re-running a training command reproduces the committed metrics bit-for-bit (same val AUROC to full float precision); only the git hash and wall-clock fields differ.

12. Limitations and honest negatives

1. **A4 beats A7 on real data** (0.740 vs 0.631). The gate's inductive bias only pays when metadata predicts reliability; on the current RAGTruth slice it does not. Planned follow-ups: the full 1,933-source ceiling (one overnight build, no API cost), scoring all folds to tighten CIs, and a subtle-vs-evident label split.
2. **Generator conditioning is weak** even on synthetic data: A6 (generator only) reaches 0.640 numeric vs A5's 0.834 (type only). The inverted-generator effect is significant only in aggregate, not per claim type.
3. **Claim extraction is sentence-based** with span-overlap labelling — partially-faithful sentences count as hallucinated, adding label noise near span boundaries.

4. **The arithmetic checker is conservative**: it verifies only exactly parseable expressions and records everything else as `UNVERIFIABLE` (all 53 numeric claims in the first slice were unverifiable rather than guessed).
5. **HaluBench's licence** is still unverified against its repository; no HaluBench number appears anywhere until it is.
6. **Three of five planned benchmarks are structurally blocked** (Section 8.4) until a judge-panel design is approved.

13. CLI reference

```
# build signal tables (per-seed; cached; --force
        regenerates)
python -m rcgate.data.build --config <name> [--seed N]
        [--force]

# train one variant or all (writes results/summary/<tag>/
        <V>_seed<N>/)
python -m rcgate.train --config <name> [--variant A0..A7
        | --all-variants] [--seeds N]

# full evaluation: metrics, bootstrap CIs, LOGO, gate
        heatmap
python -m rcgate.evaluate --config <name> [--bootstrap N]
        [--variants A0 A7 ...]
                          [--no-logo] [--out path.md]

# gate-weight tables for a trained checkpoint
python -m rcgate.explain --config <name> --variant A7 --
        seed 0

# glass-box demo (UI or headless)
streamlit run rcgate/demo/app.py
python -m rcgate.demo.app --config <name> --question Q --
        answer A
                          [--context C] [--other-answer
        A2] ...
```

Default seeds and bootstrap sizes come from the configs; every artefact lands under `results/summary/` with its config and git hash attached.

14. Regenerating this guide

This guide's source and PDF are kept **local only** (never committed): `docs/userguide.md` and `docs/RC-Gate_UserGuide.pdf`. Regenerate with:

```
pandoc docs/userguide.md -o docs/RC-Gate_UserGuide.pdf \
  --pdf-engine=weasyprint --toc --toc-depth=2 \
```

```
-V margin-top=2cm -V margin-bottom=2cm \
-V margin-left=2.2cm -V margin-right=2.2cm -V
      fontsize=10pt
```